Trustworthy Orchestration of an AI Software Company: Un-Gameable Evaluation, Shared-Context Single-Agent Design, and Production Safety

Mostafa Adawy
Odin Markets
tech@odinmarkets.com
Preprint — June 25, 2026

Abstract

Large language model (LLM) agents are increasingly assembled into multi-agent "software companies" that decompose a goal across specialised roles (architect, frontend, backend, DevOps). In practice these systems fail less from weak models than from *coordination and evaluation* failures: context is lost across hand-offs, agents silently *report success that did not happen*, and unsupervised automation damages live infrastructure. We present Odin-Orchestrator, a desktop multi-agent development system, and report the engineering principles that made it trustworthy. Our central contribution is an *un-gameable evaluation methodology*: (i) runtime-minted random tokens that must propagate across agent hops, making "context-was-shared" impossible to fake by string echoing; (ii) a *hidden-fault scanner* ("snake-hunter") that audits nominally green runs for masked errors; and (iii) a production-fingerprint safety guard with sandbox isolation that proves destructive operations never touch live systems. We further describe a deliberate move from naive multi-agent fan-out to a *single strong agent over a shared, append-only context pool*, a provenance-gated cache of reusable action modules, and an approval-gated scheduler for recurring work. We document two instructive incidents—a self-inflicted firewall outage and a caught self-deceiving test—and a phased acceptance gate (context, database-migration safety, product features) that the system must pass before promotion. Our experience supports the position that *measurement integrity*, not model capability, is the binding constraint for autonomous software agents.

1 Introduction

A growing body of tooling frames autonomous software development as a *company* of LLM agents: a manager decomposes a request, specialist workers implement parts, and a reviewer checks the result. The appeal is intuitive—division of labour—but field reports and recent studies converge on an uncomfortable finding: most failures are not reasoning failures of the underlying model, they are *system* failures of specification, coordination, and verification [3, 1, 2].

Three failure modes dominated our own build of such a system:

(F1) **Context loss across hand-offs.** When a task is split, each agent sees only its slice; details drift and the parts no longer compose.

(F2) **Fake success.** An agent (or a poorly designed test) reports a task as complete when it is not—the most corrosive failure, because it defeats the very mechanism meant to catch the others.

(F3) **Unsafe automation.** An agent with infrastructure access can take down production while "succeeding" at a narrow subtask.

This paper is an *experience report* grounded in a working system. Our thesis is that the binding constraint on autonomous software agents today is **measurement integrity**: unless an evaluation cannot be gamed—by the agent *or* by a well-meaning engineer tuning a test to pass—reported progress is not evidence of real progress. We make the following contributions:

- A formalisation of the *fake-success* problem and an **un-gameable propagation test** based on runtime-minted random tokens (§5), which certifies cross-agent context sharing without any string an agent could preemptively echo.
- A **hidden-fault audit** ("snake-hunter") that scans green runs for masked errors, plus an *anti-fake-success* gate that distinguishes a claim of success from evidence of it (§5).
- A **production-safety protocol**—fingerprinting plus sandbox isolation—that proves destructive database and deployment operations never mutate live systems (§5.5).
- A **shared-context single-agent architecture** with a provenance-gated module cache and an approval-gated recurring scheduler (§4), and two documented incidents that motivated them (§6).

2 Problem Formulation

We model an AI software company as a tuple $\mathcal{C} = \langle \mathcal{A}, \Pi, \Sigma, E \rangle$ where $\mathcal{A} = \{a_1, \ldots, a_n\}$ is a set of role-tagged agents, Π a planner (the "CTO"), Σ a shared state, and E an evaluator. A user goal g induces an ordered plan $\Pi(g) = (s_1, \ldots, s_m)$ of subtasks, each s_i assigned to an agent $a_{\rho(i)}$ by a role map ρ.

Context sharing. Let Σ_i be the state visible to the agent executing s_i. Naive fan-out gives $\Sigma_i = \text{brief}(s_i)$: each agent sees only its slice. We instead require a *shared,*

append-only pool

$$\Sigma_i \;=\; \sigma_0 \,\|\, \bigcup_{j<i} o_j,$$

where σ_0 is the verbatim goal plus the planner's decomposition and o_j is the output produced at step j. Each agent appends its result, $o_i = a_{\rho(i)}(\Sigma_i)$, so the pool grows monotonically and every agent inherits the full history. This is the formal statement of (F1)'s remedy.

The evaluation problem. For a subtask s with acceptance predicate ϕ_s, let r be an agent's *report* and w the *world effect* actually produced. Define

$$\mathrm{claim}(r) = \mathbb{1}[r \text{ asserts success}], \quad \mathrm{truth}(w) = \mathbb{1}[\phi_s(w)].$$

Fake success is the event $\mathrm{claim}(r) = 1 \wedge \mathrm{truth}(w) = 0$. A trustworthy evaluator E must satisfy two properties.

Property 1 (Soundness against fakery)**.** *E accepts s only if* $\mathrm{truth}(w) = 1$*; in particular it never accepts on the basis of* $\mathrm{claim}(r)$ *alone.*

Property 2 (Un-gameability)**.** *There is no fixed output $r^\star$, constructible without actually performing s, that causes E to accept. Formally, for every $r^\star$ independent of w,* $\Pr[E(r^\star) = \mathit{accept}]$ *is negligible.*

Property 2 is the crux. A test that scans the transcript for an expected string (e.g. "the API returns 200") violates it, because an agent can emit that string without doing the work—and, worse, an engineer can *tune* the agent to emit it, manufacturing a passing test. Our token construction (§5) is designed to make $r^\star$ impossible to guess.

Safety. Let H be a fingerprint of the production system (schema hash, row counts, service reachability). An operation θ is *production-safe* iff $H_{\text{before}} = H_{\text{after}}$ and all public service ports remain reachable throughout. The evaluator must *prove*, not assume, safety.

3 Related Work

Multi-agent orchestration. Anthropic's orchestrator–workers and evaluator–optimizer patterns [2] recommend multi-agent decomposition primarily for *parallelisable* sub-problems, and pair generation with an explicit critique loop. We adopt the evaluator–optimizer loop but argue against multi-agent fan-out for inherently sequential development work.

Single continuous agents. Cognition (Devin) argue that fragile context-passing makes long-horizon engineering better served by a *single continuous thread* that carries full, curated context rather than summaries [1]. Our shared-pool design (§4) is a concrete realisation of that position, with the "role" demoted from a separate brain to a *task tag* on one agent.

Why multi-agent systems fail. The MAST taxonomy [3] attributes the majority of multi-agent failures to specification and inter-agent misalignment (including context loss and premature or incorrect termination) rather than to the base model. Our (F1) and (F2) correspond directly to these categories; our evaluation methodology targets them.

Evaluation and "LLM-as-judge". A large literature uses an LLM to grade another LLM. Such judges are susceptible to sycophancy and to superficial cues, i.e. they can be gamed in the sense of Property 2. Our contribution is orthogonal: rather than a smarter judge, we construct *verifiable artefacts* (random tokens, fingerprints) whose presence is causally tied to the work having occurred.

4 System Architecture

ODIN-ORCHESTRATOR is a desktop application: an Electron/React frontend, a Python (FastAPI) backend, and an LLM command-line engine invoked one-shot per turn. The orchestration layer—not the model—is where our contributions live.

4.1 Shared context pool

The pool is a typed, durable object created from the user's verbatim goal and the planner's decomposition. Three operations define it: `append(who, text)`, `pool_text(limit)` (the curated view handed to the active agent), and `reconcile(delivered)`, an omission guard that checks every extracted requirement against what was actually produced. Each agent's prompt *leads* with the pool, so the "big picture" is never summarised away. This implements Σ_i of §2.

4.2 Single strong agent, role as a tag

Rather than n independent agents, one capable agent executes each step with the full pool, tagged by the role required ($\rho(i)$). This eliminates the serialization/deserialization of context that the literature identifies as the dominant failure surface, while retaining specialised behaviour through the role tag and role-specific tooling.

4.3 Provenance-gated module cache

A *Fast-Actions Library* stores reusable, parameterised scripts (e.g. a generic cloud-VM deploy/ops module) under per-role folders with sidecar metadata (`summary`, `inputs`, `status`, `provenance`). On a new task the *role* agent fetches its folder, reads the metadata, and *decides* whether a module matches—configuring and reusing it, or working from scratch. A strict rule governs growth:

Principle 1 (Earned cache)**.** *A module enters the library* only after *it has succeeded on a real task, then is generalised and documented. The library is never pre-populated with unproven "filler".*

This makes the cache a record of *demonstrated* competence, not aspiration, and keeps every entry auditable via

its provenance.

4.4 Approval-gated recurring scheduler

Projects may be *periodical*. A lightweight runner computes the next due time per schedule. Crucially, recurring runs are *not* executed unattended: when a task is due *and the operator's device is online*, the system raises an approval request; the operator approves (run) or skips (roll forward). If the due moment was missed because the device was offline (overdue beyond a grace window), the task rolls silently to the next slot. This keeps a human in the loop precisely where autonomy is riskiest, while never firing stale work.

5 Evaluation Methodology

This section is the core of the paper: how we make acceptance satisfy Properties 1 and 2.

5.1 Un-gameable propagation tokens

To certify (F1)—that context truly crosses hops—we mint a fresh random token per run, $t \sim \mathrm{Uniform}(\{\texttt{ODIN-}\Sigma^6\})$ over an alphanumeric alphabet Σ, giving $\approx 36^6 \approx 2{\times}10^9$ possibilities. The token is introduced at agent a_1 under a labelled slot and must reappear, unchanged, in the output of a downstream agent a_k that only receives it *through the pool*:

$$\text{reuse} \;=\; \mathbb{1}\big[t \in o_1 \wedge t \in o_k \wedge \text{label-bound}\big].$$

Because t is sampled at runtime, no fixed transcript $r^\star$ can contain it (Property 2); reuse is possible *only* if the pool actually carried the value. We extract tokens by their explicit labels (not by matching any `ODIN-` substring) so that a quoted *instruction* cannot be mistaken for the minted value—a subtlety that caused an early false positive (§6).

5.2 The snake-hunter: auditing green runs

A run can be "green" at the top level while hiding faults (a swallowed exception, a `command not found`, a silent timeout). The snake-hunter scans the full transcript for danger patterns and reports any found even when the run claims success:

$$\text{snakes}(w) = \{\, p \in \mathcal{D} : p \text{ matches } \text{trace}(w) \,\} \setminus \mathcal{B},$$

where $\mathcal{D}$ is a danger lexicon and $\mathcal{B}$ a benign allow-list (e.g. the literal "0 failed" in a summary line) added to suppress false positives. *Fake success* is then flagged precisely when $\text{claim}(r){=}1 \wedge \text{snakes}(w){\neq}\emptyset$, operationalising Property 1.

5.3 Actions, not mentions

A recurring temptation is to score an agent for *mentioning* the right step. We instead measure *usage*: the evaluator checks that the operation was invoked and produced the required world effect, not that the transcript discusses it.

5.4 The phased acceptance gate

Promotion is gated by a single test with independent parts; all must pass and **later phases never relax earlier ones**:

- **Phase 1 — Context.** A three-agent flow with two token hops and a typed hand-off; pool growth, token reuse, and a clean snake scan are all required.
- **Phase 2 — Migration safety.** A database migration (`deploy` then `reset`) is exercised on a *throwaway* role and database, behind a guard that aborts unless the connection string targets the sandbox; production is fingerprinted before and after.
- **Phase 3 — Product features.** Project-type classification with a recommendation engine, operational (non-technical) agents with normal/theoretical modes, and the periodical run gate (*due→pending→approve/skip*, plus missed-while-offline roll-forward) are checked by construction, with all test artefacts created and then deleted so the suite never pollutes real data.

5.5 Sandbox isolation

Every test that could touch infrastructure is redirected to an isolated sandbox via an environment override; real production deployment is refused unless an explicit allow-flag is set. Database tests create a dedicated throwaway role and database through the superuser, never the application role, and drop them afterward. The production fingerprint H is captured before and after; a divergence aborts and is reported as a safety failure, satisfying the safety predicate of §2.

6 Results and Case Studies

We report qualitative outcomes verified at the time of writing; quantitative throughput results from the timing harness are being collected and will be added in the camera-ready (§8).

6.1 The phased gate

The Phase-1 context test passes with genuine cross-hop token reuse, a typed hand-off, monotonic pool growth, and a clean snake scan. The Phase-2 migration runs on the throwaway database with the production fingerprint unchanged. The Phase-3 feature checks pass and leave *zero* residual test data. Critically, the gate distinguishes a *massaged* pass from a real one (next).

6.2 Case study: a caught self-deceiving test

An early "detail-propagation" check scanned the transcript for specific expected strings. We could make it pass by instructing the agent to echo those exact strings—a textbook violation of Property 2. We recorded this as a *gamed* pass and replaced the measurement with runtime tokens; the relabelled check now certifies brief fi-

delity without any echo-able target. The lesson generalises: *a test you can satisfy without doing the work is not a test.*

6.3 Case study: a self-inflicted production outage

While hardening firewall rotation, a rule set defaulted to deny and left only SSH open, taking public web ports (80/443) offline. Diagnosis confirmed the local services were healthy but externally blocked. The fix had three parts: (i) immediate restoration of 80/443; (ii) a rewritten rotation that opens the web ports *first* and idempotently, then manages *only* SSH; and (iii) a codified invariant—"the public web ports are sacred"—enforced in the rotation logic and an additive-only recovery script. This incident is why our safety predicate (§2) includes continuous port reachability, not just data integrity.

6.4 Token false-positive and its fix

A placeholder of the same shape as a real token (`ODIN-XXXXXX`) was matched by a permissive regex, so the test read the placeholder instead of the minted value and reported *no* reuse when reuse had in fact occurred. Switching to label-bound extraction removed the collision. We include this to underline that un-gameable tests must also be *precise*: a sound idea with a sloppy matcher yields both false negatives and exploitable false positives.

7 Discussion

Measurement integrity as the binding constraint. Across every incident, the decisive factor was not model capability but whether our *measurements* were honest. The single highest-leverage practice was adversarial scepticism toward our own green checkmarks.

When to prefer one agent. For inherently sequential engineering, a single agent over a shared pool avoided the serialization failures that the literature attributes to fan-out. We still expect genuine parallelism (independent sub-problems) to benefit from multiple workers under an orchestrator; our claim is scoped to sequential, context-heavy work.

Human-in-the-loop where it matters. The approval-gated scheduler and the universal approval gate for infrastructure-changing actions concentrate human attention on the riskiest decisions while leaving low-risk work autonomous.

8 Limitations and Threats to Validity

This is a single-system experience report; our quantitative comparison (cached vs. non-cached task time) is pending and the qualitative results, while reproducible in our gate, are not yet a controlled multi-system study. The danger lexicon of the snake-hunter is heuristic and may miss novel failure strings; the benign allow-list trades a small false-negative risk for far fewer false positives. The token test certifies *propagation*, not semantic correctness of the work; it is necessary, not sufficient, and is paired with the omission guard and feature checks for coverage.

9 Conclusion

We argued, from the experience of building a working multi-agent development system, that the binding constraint on autonomous software agents is measurement integrity. We gave a formal account of fake success and an un-gameable evaluation methodology—runtime tokens, a hidden-fault scanner, and a production-safety protocol—together with a shared-context single-agent architecture, a provenance-gated module cache, and an approval-gated scheduler. Our incidents show that the same scepticism that catches a lying *agent* must be turned on one's own *tests*. A red test that finds a real bug is a better outcome than a green test that was massaged to pass.

References

[1] Cognition AI. *Don't Build Multi-Agents*. 2024. `https://cognition.ai/blog/dont-build-multi-agents`.

[2] Anthropic. *Building Effective Agents*. 2024. `https://www.anthropic.com/research/building-effective-agents`.

[3] M. Cemri et al. *Why Do Multi-Agent LLM Systems Fail?* A Multi-Agent System Failure Taxonomy (MAST). 2025.

[4] S. Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023.

[5] N. Shinn et al. *Reflexion: Language Agents with Verbal Reinforcement Learning*. NeurIPS 2023.

[6] C. Jimenez et al. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR 2024.

[7] L. Zheng et al. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS 2023.